

Advisory Text Is Not Ground Truth: Reporter Coverage, Campaign Duplication, and a Null Result for Package Naming in GHSA/OSV Malware Advisories

Anonymous Author(s)
Anonymous Institution(s)

Abstract—Advisory databases such as GHSA and OSV are increasingly mined as ground truth for malware behaviour in package ecosystems. We collect 994 confirmed-malicious PyPI and npm advisory records and show that they correspond to 498 packages and only 317 independent campaigns (npm 152, PyPI 165) once cross-source re-reporting, multi-name campaigns and generic reporter templates are handled. We test whether compound/generic names (e.g. `py-*`, `*-tools`) go with a described payload mechanism. Pooled, the odds ratio (OR) is 1.045 [0.635, 1.721]; adjusted for ecosystem and reporter coverage, it is 0.741 [0.424, 1.294]. With 0.78 power at OR 2.0, the data rule out associations of about $OR \geq 1.7$ in the positive direction, not only large ones. Reporter coverage predicts the label far better than naming: Amazon Inspector coverage carries an OR of about 23. Two annotators validated the labeler ($\kappa = 0.78$ and 0.66): precision is 0.917 [0.819, 0.964], but it misses a described mechanism in 0.425 [0.285, 0.578] of sampled negatives; corrected for this, the OR is 1.089 [0.637, 1.869]. A GHSA-only signal seen in a smaller earlier snapshot does not replicate (OR 1.541, $p = 0.096$). The naming grammar flags only 5% of 139 real LLM-hallucinated names versus 16–21% of popular benign packages, so it does not measure hallucination-proneness. Corpus and code are released.

Index Terms—software supply chain, malicious packages, security advisories, PyPI, npm, measurement validity

I. INTRODUCTION

Malicious packages account for 49% of vulnerability reports in npm and 14% in PyPI [11], and LLM code assistants add a new route to them: a model recommends a package that does not exist, and an attacker registers the name [1], [4]. Measurement work on this threat increasingly treats the GitHub Security Advisory database (GHSA) and OSV as ground truth, both for which packages are malicious and for what the malware does.

This paper checks two assumptions that such work usually makes and rarely tests. First, that advisory records are independent observations. They are not: GHSA and OSV re-report the same package under different identifiers, and one attacker squats many names with one payload. In our corpus, 994 records reduce to 317 campaigns. Second, that a mechanism mentioned in the advisory text is a property of the malware. It is partly a property of whoever wrote the advisory: which analysts covered a package changes whether a mechanism is

described at all. Once both are accounted for, the association we test is absent, and the sample is large enough to bound it.

Research question. Among confirmed-malicious packages, is compound/generic naming associated with a described payload mechanism, after accounting for how the advisory was reported?

Contributions.

- 1) A frozen, checksummed, campaign-level corpus (994 records $\rightarrow$ 498 packages $\rightarrow$ 317 campaigns) and a reusable two-stage collapse that never keys on generic reporter templates.
- 2) Evidence that mechanism labels mined from advisory text depend strongly on reporter coverage.
- 3) A two-annotator labeler validation measuring precision and misses, and a misclassification-corrected estimate.
- 4) A null for naming $\rightarrow$ mechanism that excludes moderate effects, and a negative validation of the naming grammar as a hallucination proxy.

II. BACKGROUND AND RELATED WORK

Package hallucination. Spracklen et al. [1] measured hallucinated package names across 16 code-generating models and found the problem persistent across models. Churilov [9] replicated the study on 2026 frontier models and released the set of names hallucinated by all five models tested, which we use in Section III-C. Huang et al. [4] registered hallucinated npm names and observed real installs, which establishes the threat model behind this work.

Malicious-package measurement. Akhavani et al. [11] analyse vulnerability reports across ten language ecosystems and find that malicious packages make up a large share of reported npm and PyPI vulnerabilities. Guo et al. [10] benchmark npm malicious-package detectors on a large labelled set. Hillah et al. [3] use PyPI registration metadata to give a calibrated posterior for registered-but-suspicious packages. Guo et al. note that typosquatting campaigns reuse identical payloads across many names, but treat each package as a separate instance; Akhavani et al. remove 6,391 duplicate reports and prefer Snyk text over GitHub advisories because it is more detailed, a reporter-coverage choice whose effect on estimates they do not measure. We quantify both.

Scope. Work on hallucinated agent-skill names that host adversarial prompts [2] concerns a different artifact and a different mechanism, and is not studied here.

III. DATA AND METHOD

Fig. 1 shows the pipeline.

A. Collection

We query the GHSA API for advisories of type `malware` in the pip and npm ecosystems, newest first, following the API’s cursor until each ecosystem has 250 advisory–package records, and query OSV.dev for each package found. After removing exact duplicates on (ecosystem, package, advisory id), the corpus has 994 records (500 GHSA, 494 OSV) for 498 packages (250 PyPI, 248 npm). The pull was made on 5 October 2026. Its GHSA advisories were published between 21 July and 5 October 2026; npm publishes so many malware advisories that its half spans only 30 September to 5 October. The pull is frozen to a snapshot with a SHA-256 manifest, and every number in this paper is computed from that file.

B. Campaign collapse

Fisher’s and CMH tests assume independent units, and one npm family alone has 74 member packages. We collapse in two stages. Stage 1 merges records of the same (ecosystem, package), which catches GHSA/OSV re-reports. Stage 2 groups packages by a campaign signature, checked in priority order:

- 1) the reporter’s own `Campaign:` identifier, when present (the kam193 reporter assigns these);
- 2) otherwise the first 200 characters of the description, after stripping `## Source:` headers, masking the package’s own name and masking URLs.

Generic texts are never used as keys, because they are identical across unrelated incidents: GHSA’s “should be considered fully compromised” boilerplate, kam193’s generic install-time pentest template, Amazon Inspector’s one-line “was found to contain malicious code” verdict, and advisories whose only text is an OpenSSF credit link. Packages carrying only such text stay as their own unit. A unit counts as grammar-flagged, or as DV-positive, if any member does, since names within one campaign can differ in their grammar flag. An audit of near-identical signatures motivated the last two template rules and the URL mask.

The result is 498 packages $\rightarrow$ 317 campaigns (npm 152, PyPI 165), of which 277 have a single member. The largest are two WhatsApp-library fork families of 74 and 15 npm names and an 11-name PyPI campaign.

C. Independent variable: compound/generic naming

A name is flagged if it contains a token from lists of compound suffixes, compound prefixes or trend suffixes (e.g. `py-`, `-tools`, `-client`, `-sdk`). The classifier reads the name only. The grammar was designed as a proxy for names an LLM might hallucinate, so we tested it against real hallucinations (Table I). It flags 7 of the 139 names that all five models

TABLE I
GRAMMAR FLAG RATE BY SET, WITH WILSON 95% CIs.

Set	<i>n</i>	Flagged	Rate [95% CI]
Hallucinated names [9]	139	7	0.050 [0.025, 0.100]
Benign top-5000 PyPI	5,000	821	0.164 [0.154, 0.175]
Benign top-5000 npm	5,000	1,031	0.206 [0.195, 0.218]
Malware campaigns	317	85	0.268 [0.222, 0.319]

in Churilov’s study [9] hallucinated, against 16–21% of the top-5,000 benign packages. We therefore describe the variable as compound/generic naming, which is what it measures. The benign sets must be chosen with care: an npm control drawn from search-API results for common terms is flagged at 73%, because the search terms are themselves grammar tokens. Flagged hallucinated names include, e.g., `aws-cdk` and `flask-idempotency`.

D. Dependent variable: described mechanism

A first taxonomy, aimed at agent-directed prompt injection, found nothing: advisory text is third-person analyst narration, not text from the package. The dependent variable is therefore whether the advisory text describes at least one of six mechanisms:

- obfuscated payload execution;
- a hidden install hook;
- credential or wallet exfiltration;
- remote backdoor access;
- remote payload retrieval;
- a brand-impersonation narrative.

The patterns were first derived from PyPI text and missed npm phrasing; adding `postinstall` hooks, JavaScript obfuscators and verbs such as *reads* and *POSTs* fixed this. A negation guard drops a match when a cue (*does not*, *do not*, *did not*, *doesn’t*, *no evidence of*) occurs within 40 characters before it, unless *but*, a semicolon or a period intervenes. The labeler’s signature accepts text only, never the name, and a test enforces this. The text is the union of a package’s GHSA and OSV descriptions; GHSA-only text is a sensitivity analysis (Section IV-E).

Validation. Two authors annotated independently, then adjudicated, under a written rule: a match counts only if prose, not an analyst tag list, describes the mechanism. Seeded samples stratified by ecosystem $\times$ Amazon Inspector coverage gave 60 labeler-positive and 40 labeler-negative campaigns (full text). First-pass κ was 0.783 on positives (96.7% raw agreement) and 0.658 on negatives (82.5%). Precision is 55/60 = 0.917 [0.819, 0.964]. All five false positives are the same kam193 tag-list phrase (“Downloads and executes a remote executable”), so remote payload retrieval has precision 4/9 while every other category is at 1.00. The miss rate (negatives whose prose does describe a mechanism) is 17/40 = 0.425 [0.285, 0.578], or 14/40 = 0.350 [0.221, 0.505] counting three borderline rows as correct. Misses concentrate in npm campaigns with Amazon Inspector text (11 of 15 sampled).

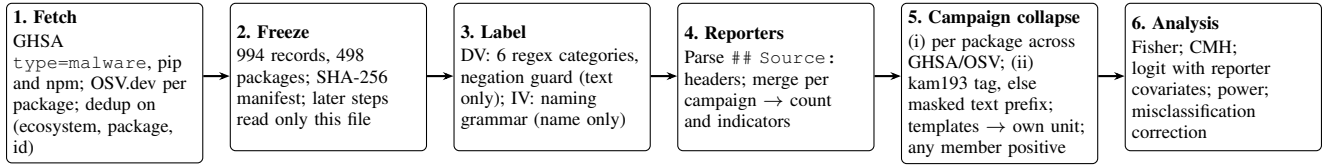


Fig. 1. Data collection, labeling, and analysis pipeline. The DV and IV labelers in step 3 read disjoint inputs, so the association test is not circular.

Sending only host fingerprints (hostname, IP, username) fits no category.

E. Reporter coverage

We parse the `## Source:` headers in both texts of every package and merge them per campaign. GHSA’s boilerplate and OSV’s `ghsa-malware` section are the same text and count as one reporter. Headerless text without boilerplate is *unattributed* (96 of 498 packages). From the parsed headers we derive a reporter count (`n_reporters`) and three indicators: `kam193`, `amazon-inspector` and `boilerplate`.

F. Analysis

We use Fisher’s exact test on the pooled and per-ecosystem 2×2 tables, and CMH tests stratified by ecosystem and by ecosystem $\times$ reporter count [6]. We fit the logistic regression $DV \sim grammar + ecosystem + n_reporters + amazon_inspector$ (model d), and three variants: (a) without the Amazon term, (b) without the reporter count, and (c) (a) plus the boilerplate and `kam193` indicators. ORs come with Woolf, Robins–Breslow–Greenland or Wald 95% CIs as appropriate. Power at the achieved n follows Hsieh et al. [7]. We correct the pooled OR for outcome misclassification by predictive-value back-calculation [8]: per grammar row, true positives = positives $\times$ precision + negatives $\times$ miss rate (0 for no-text campaigns). This assumes equal predictive values across rows, as the samples suggest (precision 0.95 vs. 0.90, miss rate 0.38 vs. 0.44). The interval uses 20,000 Jeffreys Beta draws plus random error.

IV. RESULTS

A. Descriptive

Of the 317 campaigns, 85 are grammar-flagged and 169 are DV-positive. The DV rate depends strongly on who reported the campaign, not on naming (Fig. 2):

- `amazon-inspector` coverage: 0.66 (162/245), against 0.10 (7/72) without;
- `kam193` coverage: 0.55 (77/139);
- `boilerplate` among the reporters: 0.29 (10/35).

Reporter coverage is not independent of naming either: 2 of the 35 boilerplate campaigns are grammar-flagged, against 83 of the other 282.

B. Main test

The pooled table is $[[46, 39], [123, 109]]$, which gives OR 1.045 [0.635, 1.721], $p = 0.899$ (Table II). Adjusting for ecosystem with CMH gives 1.046 [0.635, 1.722], $p = 0.860$.

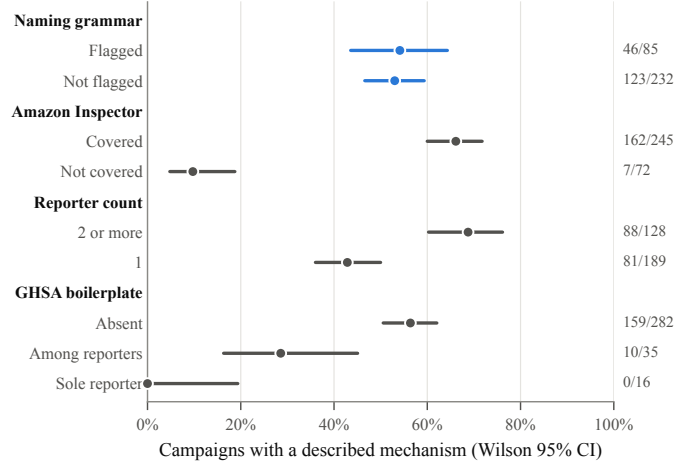


Fig. 2. Share of campaigns with a described mechanism (Wilson 95% CIs), flat across the naming split but not across reporter coverage.

C. Adjusted

In model (d) the grammar OR is 0.741 [0.424, 1.294], $p = 0.292$, with a variance inflation factor of 1.02. Reporter coverage explains the label instead. Amazon Inspector coverage has OR 23.1 in model (d) and 22.9 in model (b); entered without it, the reporter count has OR 4.05 (model a). With both terms in the model (variance inflation factors at most 1.94), the reporter count falls to 0.99, and model (d) as a whole is significant (likelihood-ratio $p < 10^{-16}$).

D. Misclassification

The corrected OR is 1.089 [0.637, 1.869], or 1.078 [0.640, 1.826] with the 14/40 miss rate: similar error rates in both rows dilute any effect only slightly.

E. Text-source sensitivity

Reading only GHSA’s text gives OR 1.541 [0.934, 2.542], $p = 0.096$ (CMH 1.557 [0.941, 2.578], $p = 0.085$). In an earlier snapshot of 105 campaigns the GHSA-only CMH estimate was 2.869 [1.085, 7.583], $p = 0.027$; that signal did not replicate. A single-source estimate depends on which reporter’s text is dropped.

F. Power

With 26.8% of campaigns flagged and a baseline DV rate of 0.53 among unflagged campaigns, the design has 0.78 power at OR 2.0, 0.99 at OR 3.0 and 0.36 at OR 1.5. Reaching 80% power at OR 2.0 needs about 335 campaigns.

TABLE II
GRAMMAR FLAG VS. DESCRIBED MECHANISM, 317 CAMPAIGNS, GHSA+OSV TEXT UNLESS NOTED. REGRESSION MODELS (A)–(D) AS DEFINED IN SECTION III-F.

Specification	<i>n</i>	Grammar OR [95% CI]	<i>p</i>	Reporter terms, OR [95% CI]	<i>p</i>
Pooled, Fisher	317	1.045 [0.635, 1.721]	0.899		
npm, Fisher	152	1.215 [0.587, 2.518]	0.712		
PyPI, Fisher	165	0.915 [0.461, 1.816]	0.862		
CMH by ecosystem	317	1.046 [0.635, 1.722]	0.860		
CMH by ecosystem $\times$ reporter count	317	0.849 [0.498, 1.445]	0.544		
Logit (d)	317	0.741 [0.424, 1.294]	0.292	n_reporters 0.987 [0.500, 1.947] amazon-inspector 23.092 [8.525, 62.551]	0.969 <0.001
Logit (a)	317	0.956 [0.568, 1.609]	0.866	n_reporters 4.047 [2.336, 7.011]	<0.001
Logit (b)	317	0.741 [0.424, 1.294]	0.292	amazon-inspector 22.863 [9.688, 53.956]	<0.001
Logit (c)	317	0.689 [0.394, 1.206]	0.192	n_reporters 12.158 [5.328, 27.746] boilerplate 0.044 [0.014, 0.139] kam193 0.379 [0.125, 1.150]	<0.001 <0.001 0.087
<i>Sensitivity: GHSA text only, Fisher</i>	317	1.541 [0.934, 2.542]	0.096		
<i>Sensitivity: GHSA text only, CMH</i>	317	1.557 [0.941, 2.578]	0.085		
<i>Sensitivity: no-text campaigns excluded</i>	306	0.974 [0.588, 1.614]	1.000		
<i>Sensitivity: 13 template packages merged</i>	305	1.122 [0.670, 1.878]	0.696		
<i>Misclassification-corrected (miss rate 17/40)</i>	317	1.089 [0.637, 1.869]			
<i>Misclassification-corrected (miss rate 14/40)</i>	317	1.078 [0.640, 1.826]			

V. DISCUSSION

What the null means. We find no evidence that compound/generic naming goes with a described mechanism. The pooled interval excludes OR above 1.72 and below 0.64, and the misclassification-corrected interval excludes OR above 1.87, so a positive association of about OR 1.7–2 or more is unlikely. Effects near OR 1.5 cannot be excluded (power 0.36); a cost-minimising attacker model [5] could motivate only such a small effect.

What the reporter finding means for others. Any study that uses GHSA or OSV text as a behavioural label inherits a reporter-coverage confound: here, one reporter’s coverage moves the label rate from 0.10 to 0.66. Dropping OSV’s text moved our estimate from 1.05 to 1.54, and to 2.44 in the earlier snapshot. We recommend using the union of the source texts, parsing reporter sections and controlling for reporter coverage.

Why we do not claim “hallucination”. The grammar flags 5% of real hallucinations, most of them import or module names used as package names (e.g. `objc`), so it measures generic naming. The gradient from 16–21% (benign) to 27% (malware campaigns) is descriptive, and the sets are not matched.

VI. THREATS TO VALIDITY

Construct. The labeler is a regex taxonomy. Two annotators validated it ($\kappa = 0.783$ and 0.658) before adjudication. Recall was measured: it misses mechanisms in about four of ten sampled negatives, mostly in Amazon Inspector’s npm prose, and host-fingerprint sending has no category; the corrected OR bounds the effect. Annotators saw package names. The grammar is validated only as a measure of generic naming, not of hallucination.

Internal. Reporter coverage is modelled but not exhausted: 96 of 498 packages are unattributed. Grouping rules were revised as audits surfaced merge failures (142 $\rightarrow$ 106 $\rightarrow$ 94

$\rightarrow$ 93 $\rightarrow$ 105 campaigns on the earlier snapshot, with OR 0.884, 0.964 and 0.812 at the last three), and further under-merging is possible; merging the 13 template packages leaves the estimate null.

External. GHSA advisories in the corpus were published between 21 July and 5 October 2026, and the npm half spans six days. The corpus covers two ecosystems and 317 campaigns.

Statistical. The design has 0.36 power at OR 1.5, several strata are sparse, and CMH assumes a homogeneous OR across strata.

VII. CONCLUSION AND FUTURE WORK

Advisory records are not independent, and advisory text is not ground truth. After correcting for both, compound/generic naming shows no association with described malware mechanisms at a sample size that rules out moderate effects, while reporter coverage does. Next steps are a longer npm collection window; extending the labeler to Amazon Inspector’s npm phrasing, ignoring tag lists and adding a host-fingerprint category; a matched benign comparison; and a hallucination instrument based on import names rather than naming grammar.

Artifact availability. The frozen snapshot, manifests, annotation sheets, analysis code and tests are available at [anonymized]. GHSA advisory data are licensed CC BY 4.0, OSV records carry per-source licences, and the hallucinated-name set [9] is CC BY 4.0.

ACKNOWLEDGMENTS

The authors used an AI assistant (Claude, Anthropic) for help with code and drafting. The authors reviewed all code, analyses and text and take full responsibility for the content of this paper.

REFERENCES

- [1] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala, “We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs,” in *Proc. 34th USENIX Security Symposium*, 2025, arXiv:2406.10279.
- [2] A. Spira, S. Cohen, E. Feldman, R. Bitton, A. Wool, and B. Nassi, “Beware of agentic botnets: Scalable untargeted promptware attacks via universal and transferable adversarial HalluSquatting,” *arXiv preprint arXiv:2607.07433*, 2026.
- [3] L. M. Hillah, J.-M. Richard, and R. Hasnaoui, “Bayesian-calibrated detection of hallucinated package imports in AI-assisted code,” *arXiv preprint arXiv:2606.13918*, 2026.
- [4] Z. Huang, C. Liu, Z. Wang, X. Cui, X. Deng, and M. Xu, “Package hallucinations as phantoms in open-source software supply chains: An empirical security analysis,” *ACM Trans. Softw. Eng. Methodol.*, 2026.
- [5] L. Allodi, F. Massacci, and J. Williams, “The work-averse cyberattacker model: Theory and evidence from two million attack signatures,” *Risk Analysis*, vol. 42, no. 8, pp. 1623–1642, 2022.
- [6] A. Agresti, *Categorical Data Analysis*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.
- [7] F. Y. Hsieh, D. A. Bloch, and M. D. Larsen, “A simple method of sample size calculation for linear and logistic regression,” *Statistics in Medicine*, vol. 17, no. 14, pp. 1623–1634, 1998.
- [8] T. L. Lash, M. P. Fox, and A. K. Fink, *Applying Quantitative Bias Analysis to Epidemiologic Data*, ch. 6. New York, NY, USA: Springer, 2009.
- [9] A. Churilov, “The range shrinks, the threat remains: Re-evaluating LLM package hallucinations on the 2026 frontier-model cohort,” *arXiv preprint arXiv:2605.17062*, 2026.
- [10] W. Guo, Z. Chen, Z. Xu, C. Liu, M. Kang, S. Song, C. Liu, Y. Xu, W. Sun, and Y. Liu, “How effective are NPM malicious package detectors? A large-scale empirical study,” *arXiv preprint arXiv:2603.27549*, 2026.
- [11] S. A. Akhavani, B. Ousat, and A. Kharraz, “Open source, open threats? Investigating security challenges in open-source software,” *arXiv preprint arXiv:2506.12995*, 2025.